

Documentation of the Compilation / Execution Process

1. Lexical Analysis

The lexical analysis phase is responsible for converting the raw source code (.cl files) into a sequence of tokens.

This phase is implemented using **PLY (Python Lex-Yacc)**.

The lexer identifies:

- Keywords (WORLD, AGENT, IF, WHILE, REPORT, etc.)
- Identifiers (variable names, function names)
- Literals (integers, directions)
- Operators (+, LT, GT, EQ, AND, OR, etc.)
- Delimiters ((,), {, }, ;)

Invalid characters and malformed tokens are detected and reported with line numbers to assist debugging.

2. Parsing (Syntax Analysis)

The parser receives the token stream produced by the lexer and validates it against the Cleaning-World grammar.

The grammar defines:

- World declarations
- Agent definitions
- Function definitions
- Statements (assignments, conditionals, loops)
- Expressions with correct precedence

If the program is syntactically valid, the parser produces a **Concrete Syntax Tree (CST)**.

If syntax errors are encountered, parsing stops and appropriate error messages are generated.

3. Static Semantic Analysis

Static semantic analysis is performed on the CST before execution.

This phase checks:

- Variables are declared before use

- Duplicate variable and function declarations
- Correct number of arguments in function calls
- Type consistency in expressions
- Valid use of control structures

If semantic errors are found, execution is stopped and a list of errors is returned.

4. AST Construction

After successful semantic analysis, the CST is transformed into an **Abstract Syntax Tree (AST)**.

The AST:

- Removes syntactic details that are not needed for execution
- Represents the logical structure of the program
- Is used directly by the interpreter

Each language construct corresponds to a specific AST node.

5. Direct Interpretation (Part 5)

For Part 5, the project uses **direct interpretation from the AST**.

Execution process:

- The interpreter traverses the AST node by node
- Statements and expressions are evaluated dynamically
- A stack-based environment is used to handle function calls
- Built-in robot commands are executed directly

Design simplifications:

- No recursion
- No non-local references
- Single call stack

This approach allows full execution without generating intermediate code.